



འབྲུག་རྒྱལ་ཁོངས་གཞིའི་སྒྲིག་ཁག་གི་སྒྲིག་ཁོངས་།

College of Science and Technology
Rinchending: Bhutan

WEB101
Web Application Fundamentals
(SS2026)

Practical 7 Report

Submitted By;

Name: Khamsum Yoezer

Student No.: 02250390

Programme: BE 1SWE

Date: 18/5/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication:** the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration:** making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data:** presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism:** the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Aim

The aim of this practical exercise is to implement different charting libraries in a React application to create an analytics dashboard. The practical focuses on displaying data visually using multiple chart types such as line charts, pie charts, bar charts, and area charts. The exercise also aims to help students understand responsive and interactive data visualization techniques using modern React charting libraries.

Theory

Data visualization is the graphical representation of information and data using visual elements such as charts, graphs, and dashboards. It helps users understand trends, comparisons, and patterns easily.

In modern web development, charting libraries are commonly used to build dashboards and analytics systems. React provides many powerful charting libraries that simplify the creation of interactive and responsive charts.

The following chart types were implemented in this practical:

1. **Line Chart**

A line chart is used to display trends over time. In this practical, it was used to represent monthly sales data.

2. **Pie Chart**

A pie chart displays proportional data in a circular form. It was used to show product category distribution.

3. **Bar Chart**

A bar chart compares data across categories. It was used to visualize customer acquisition data.

4. **Area Chart**

An area chart is similar to a line chart but fills the area beneath the line. It was used to display weekly visitor statistics.

The practical mainly used:

- React.js
- Recharts Library
- react-chartjs-2
- Chart.js

These libraries provide reusable components for creating responsive and interactive visualizations.

Implementation Steps

Step 1: Project Setup

1. Cloned the provided GitHub repository:

`git clone https://github.com/syangche/Data-Visualisation.git`

2. Opened the project in Visual Studio Code.
3. Installed the required dependencies:

```
PS C:\Users\Dell\OneDrive\Desktop\WEB101_02250390_Practical07\Data-Visualisation>npm install
added 267 packages, and audited 268 packages in 24s

49 packages are looking for funding
  run `npm fund` for details

16 vulnerabilities (2 low, 6 moderate, 8 high)

To address all issues, run:
  npm audit fix
```

4. Run ``npm audit`` for details.

Installed charting libraries:

```
PS C:\Users\Dell\OneDrive\Desktop\WEB101_02250390_Practical07\Data-Visualisation>npm install recharts chart.js react-chartjs-2
changed 3 packages, and audited 268 packages in 19s

49 packages are looking for funding
  run `npm fund` for details

16 vulnerabilities (2 low, 6 moderate, 8 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\Dell\OneDrive\Desktop\WEB101_02250390_Practical07\Data-Visualisation>
```

Step 2: Creating Monthly Sales Chart

A line chart was created using Recharts.

File Created:

components/MonthlySalesChart.jsx

Features:

- Displays monthly sales performance
- Responsive design
- Uses LineChart component from Recharts

```

import {
  LineChart,
  Line,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer,
  ReferenceLine
} from 'recharts';
import { monthlySales } from '../data/salesData';

Tabnine | Edit | Test | Explain | Document
export default function MonthlySalesChart() {
  return (
    <ResponsiveContainer width="100%" height="100%">
      <LineChart
        data={monthlySales}
        margin={{ top: 10, right: 30, left: 0, bottom: 0 }}
      >
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="month" />
        <YAxis />
        <Tooltip formatter={(value) => `$$${value.toLocaleString()}`} />
        <Legend />
        <Line type="monotone" dataKey="sales" stroke="#8884d8" strokeWidth={2} name="Sales" />
        <Line type="monotone" dataKey="profit" stroke="#82ca9d" strokeWidth={2} name="Profit" />
        <Line type="monotone" dataKey="target" stroke="#ff7300" strokeDasharray="5 5" name="Target" />
      </LineChart>
    </ResponsiveContainer>
  );
}

```

Step 3: Creating Product Category Chart

A pie chart was implemented using Recharts.

File Created:

components/ProductCategoryChart.jsx

Features:

- Shows product category distribution
- Uses PieChart component
- Includes labels and colors

```

import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip, Legend } from 'recharts';
import { productSales } from '../data/salesData';

const COLORS = ['#0088FE', '#00C49F', '#FFBB28', '#FF8042', '#8884D8'];

Tabnine | Edit | Test | Explain | Document
export default function ProductCategoryChart() {
  return (
    <ResponsiveContainer width="100%" height="100%">
      <PieChart>
        <Pie
          data={productSales}
          cx="50%"
          cy="50%"
          labelLine={false}
          outerRadius={80}
          fill="#8884d8"
          dataKey="value"
          label={({ name, percent }) => `${name} ${(percent * 100).toFixed(0)}%` }
        />
        {productSales.map((entry, index) => (
          <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
        ))}
      </PieChart>
      <Tooltip formatter={(value) => `${value}%` } />
      <Legend />
    </ResponsiveContainer>
  );
}

```

Step 4: Creating Customer Acquisition Chart

A bar chart was implemented using react-chartjs-2.

File Created:

components/CustomerAcquisitionChart.jsx

Features:

- Displays customer acquisition statistics
- Uses Bar chart component
- Interactive chart rendering

```

import { useState, useEffect } from 'react';
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend
} from 'chart.js';
import { format, parseISO } from 'date-fns';
import { customerData } from '../data/salesData';

ChartJS.register(CategoryScale, LinearScale, BarElement, Title, Tooltip, Legend);

Tabnine | Edit | Test | Explain | Document
export default function CustomerAcquisitionChart() {
  const [chartData, setChartData] = useState({ labels: [], datasets: [] });

  useEffect(() => {
    const labels = customerData.map(item => format(parseISO(item.date), 'MMM yyyy'));
    setChartData({
      labels,
      datasets: [
        {
          label: 'New Customers',
          data: customerData.map(item => item.newCustomers),
          backgroundColor: 'rgba(53, 162, 235, 0.7)',
        },
        {
          label: 'Returning Customers',
          data: customerData.map(item => item.returningCustomers),
          backgroundColor: 'rgba(255, 99, 132, 0.7)',
        },
      ],
    });
  });
}

```

Step 5: Creating Weekly Visitors Chart

An area chart was created using react-chartjs-2.

File Created:

components/WeeklyVisitorsChart.jsx

Features:

- Shows weekly visitor data
- Responsive visualization
- Filled chart area for better readability


```

import { useState, useEffect } from 'react';
import { Line } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  PointElement,
  LineElement,
  Title,
  Tooltip,
  Filler,
  Legend,
} from 'chart.js';
import { weeklyVisitors } from '../data/salesData';

ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip, Filler, Legend);

Tabnine | Edit | Test | Explain | Document
export default function WeeklyVisitorsChart() {
  const [chartData, setChartData] = useState({ labels: [], datasets: [] });

  useEffect(() => {
    const labels = weeklyVisitors.map(item => `Week ${item.week}`);
    setChartData({
      labels,
      datasets: [
        {
          fill: true,
          label: 'Weekly Visitors',
          data: weeklyVisitors.map(item => item.visitors),
          borderColor: '■ rgb(75, 192, 192)',
          backgroundColor: '■ rgba(75, 192, 192, 0.2)',
          tension: 0.4
        }
      ]
    });
  }, []);
}

```

Step 6: Updating App.jsx

All implemented chart components were imported and displayed in the main application file.

File Updated:

App.jsx

Tasks Performed:

- Imported all chart components
- Arranged charts in the dashboard layout
- Rendered charts successfully in the browser

```
import './App.css';
import MonthlySalesChart from './components/MonthlySalesChart';
import ProductCategoryChart from './components/ProductCategoryChart';
import CustomerAcquisitionChart from './components/CustomerAcquisitionChart';
import WeeklyVisitorsChart from './components/WeeklyVisitorsChart';
```

Tabnine | Edit | Test | Explain | Document

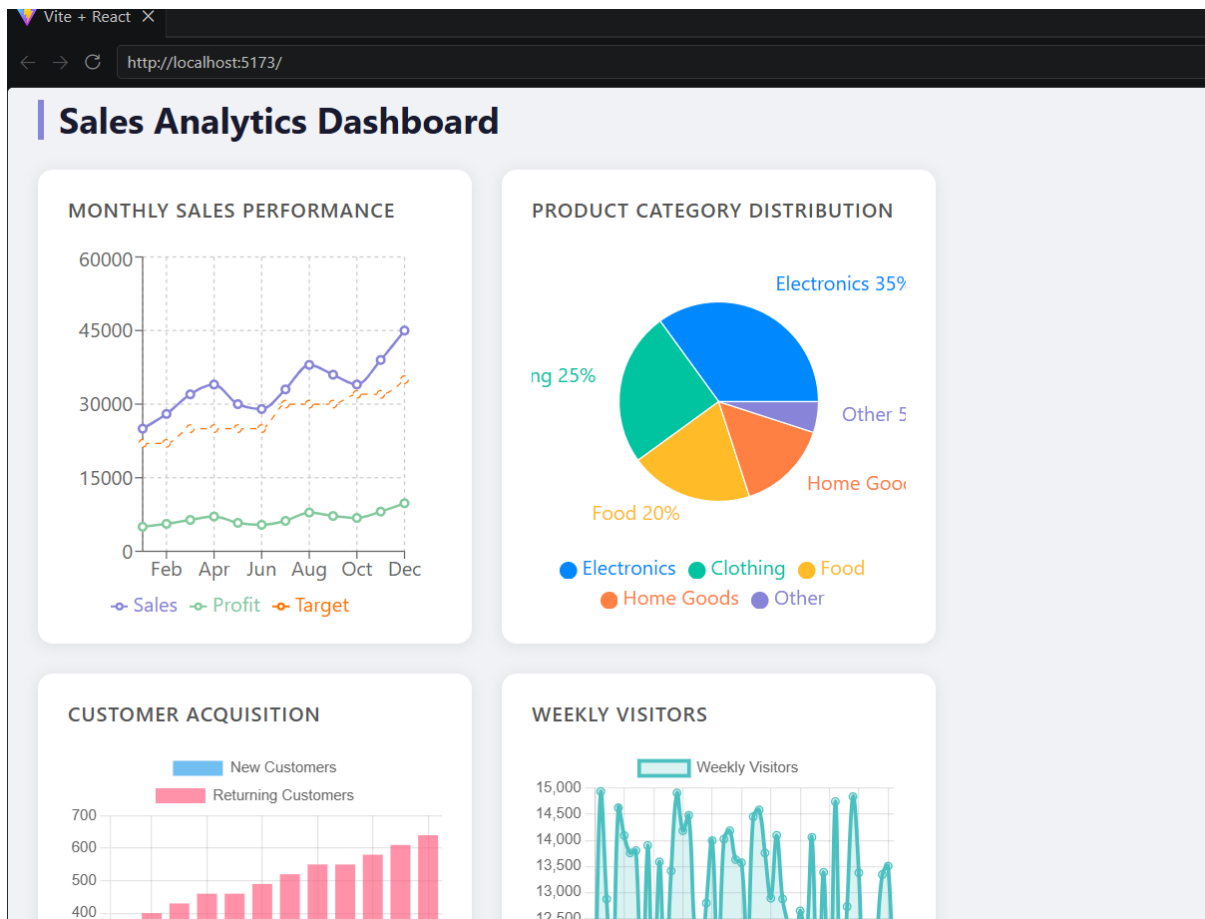
```
export default function App() {
  return (
    <main className="main">
      <h1>Sales Analytics Dashboard</h1>
      <div className="grid">
        <section className="card">
          <h2>Monthly Sales Performance</h2>
          <div className="chartContainer">
            <MonthlySalesChart />
          </div>
        </section>
        <section className="card">
          <h2>Product Category Distribution</h2>
          <div className="chartContainer">
            <ProductCategoryChart />
          </div>
        </section>
        <section className="card">
          <h2>Customer Acquisition</h2>
          <div className="chartContainer">
            <CustomerAcquisitionChart />
          </div>
        </section>
        <section className="card">
          <h2>Weekly Visitors</h2>
          <div className="chartContainer">
            <WeeklyVisitorsChart />
          </div>
        </section>
      </div>
    </main>
  );
}
```

Output

The final application displayed:

- Monthly Sales Line Chart
- Product Category Pie Chart
- Customer Acquisition Bar Chart
- Weekly Visitors Area Chart

All charts were responsive and interactive, providing a complete analytics dashboard interface.



Conclusion

In this practical, different charting libraries were successfully implemented in a React application to create an analytics dashboard. Various chart types including line charts, pie charts, bar charts, and area charts were created and integrated into the application. The exercise improved understanding of data visualization concepts, React component structure, and the use of charting libraries for presenting analytical data effectively. The dashboard was responsive, interactive, and visually informative.

Reference

1. Chart.js. (n.d.). *Chart.js documentation*. Retrieved May 17, 2026, from <https://www.chartjs.org/docs/latest/>

2. Meta Platforms, Inc. (n.d.). *React documentation*. Retrieved May 17, 2026, from <https://react.dev>
3. react-chartjs-2. (n.d.). *react-chartjs-2 documentation*. Retrieved May 17, 2026, from <https://react-chartjs-2.js.org/>
4. Recharts Group. (n.d.). *Recharts documentation*. Retrieved May 17, 2026, from <https://recharts.org>
5. Yangchen, S. (n.d.). *Data-Visualisation* [GitHub repository]. GitHub. Retrieved May 17, 2026, from <https://github.com/syangche/Data-Visualisation.git>